

PRESTE: Preserving Tiny Exponent Precision for Efficient Sub-8-bit LLM Inference and Fine-Tuning

Jehun Lee, Hyesung Jeon, Juchan Lee, and Jae-Joon Kim

Seoul National University, Republic of Korea

{jehun.lee,hjeon2k,juchan.lee,kimjaejeon}@snu.ac.kr

Abstract

Fine-tuning large language models (LLMs) for specialized downstream tasks has become the prevailing paradigm in real-world deployments. Previous studies have explored 8-bit floating-point (FP8) quantization for neural network training. However, limited exponent range often results in precision loss and requires high-precision hardware accumulators to ensure resolution. We propose a novel quantization technique for fine-tuning and inference that preserves small-magnitude values, which would otherwise vanish under conventional quantization schemes. Additionally, we design a hardware architecture optimized for this format, enabling energy-efficient computation with reduced reliance on high-precision accumulators. Our design achieves $3.51\times$ and $1.89\times$ reductions in area, and improves energy efficiency by $4.34\times$ and $2.31\times$, compared to Bfloat16 and FP8-based dot-product units, respectively. Comprehensive evaluations across LLMs demonstrate that our approach achieves $2.22\text{--}3.55\times$ higher energy efficiency and $1.67\text{--}2.26\times$ higher performance compared to baseline methods during inference and fine-tuning.

1 Introduction

Pre-trained large language models (LLMs) have demonstrated remarkable capabilities across various tasks, establishing themselves as foundational models [36]. Leveraging pre-trained LLMs that encode general knowledge and subsequently fine-tuning them on task-specific datasets for domain-specific tasks, has proven to be both time-efficient and performance-enhancing compared to training models from scratch [9, 26]. Motivated by this superior performance of in downstream tasks, recent research has intensively explored the fine-tuning of transformer models to better align with domain-specific data. For applications from healthcare to personal assistants, access to sensitive datasets is indispensable to construct and deploy advanced models [30, 31]. Enabling fine-tuning and inference directly on edge devices allows locally generated data to be processed in situ, enhancing privacy and eliminating sensitive data transmission to centralized data centers.

In conventional DNN training, including LLM fine-tuning, optimizers are used to update the weight parameters. These optimizer states require substantial memory to store first and second momentum states, as well as gradients, resulting in $3\times$ memory overhead relative to the model size [13]. To address these challenges, parameter-efficient fine-tuning (PEFT) methods have been

proposed. Among these, Low-rank adaptation (LoRA) has gained prominence by significantly reducing the number of trainable parameters while freezing pretrained model, thereby decreasing the storage requirements for optimizer states [10].

However, despite these advantages, LoRA-based fine-tuning still incurs substantial off-chip memory traffic during both forward and backward passes. Fig. 1(a) illustrates the computational flow of LoRA fine-tuning. The forward pass includes general matrix multiplication (GEMM) operations between activations and weights. The backward pass similarly involves GEMM operations to compute gradients with respect to activation and weight. These high-precision GEMM operations lead to substantial memory traffic and computational resources, posing challenges to efficient fine-tuning. Fig. 1(b) presents the off-chip memory traffic when applying LoRA fine-tuning to LLaMA-2 7B model in various methods [36]. Although LoRA effectively reduces the memory overhead associated with optimizer states, the GEMM operations during fine-tuning are performed using merged weights, formed by combining the frozen weights with the LoRA adapters. Consequently, the volume of memory access traffics for activations, activation gradients, and weights remains largely unchanged, indicating that LoRA alone does not alleviate the memory traffic demands of these computations.

Moreover, edge devices inherently suffer from strict computational and memory constraints, necessitating an efficient quantization method to make feasible on-device fine-tuning [9, 26, 33]. However, most existing quantization techniques are optimized for inference workloads [2, 6, 8, 14, 18, 42]. OliVe and Tender proposed custom number formats for efficient inference [8, 18]. QLoRA applies 4-bit quantization to frozen weights, while QA-LoRA quantizes both weights and LoRA adapters [6, 41]. During the backward pass, however, the quantized weights are dequantized to BF16, and the resulting weights, activations, and activation gradients all participate in BF16 GEMM operations. Consequently, the reduction in memory traffic during fine-tuning remains limited and still requires high-precision compute resources, as shown in Fig. 1(b,c).

To alleviate memory traffic and computational cost, quantizing floating-point tensors to low-precision formats such as FP8 becomes prevalent for DNN training [23, 25, 32]. However, the limited exponent width of FP8 format results in significant numerical errors and instability, especially during training. A technique such as bias shifting, which adjusts the exponent bias through heuristic searches and profiling, is inadequate in edge systems [23, 32]. Microscaling (MX) applies group-level quantization with a shared scaling factor to avoid heuristic scaling procedures [27]. Nevertheless, 8-bit MX floating-point (MXFP8) relies on an FP32-level accumulator and FP8 multiplier unit, which results in substantial computational overhead [24]. This requirement makes MXFP8 less suitable for on-devices training under limited compute budgets.



This work is licensed under a Creative Commons Attribution 4.0 International License. ICCAD '26, San Jose, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2873-0/2026/11

<https://doi.org/10.1145/3831252.3834226>

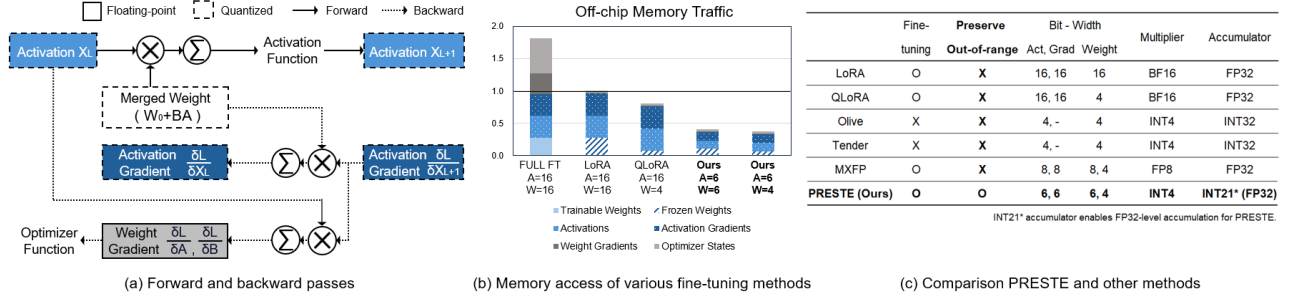


Figure 1: (a) Compute flow of forward and backward pass using quantized GEMM. (b) Normalized memory access when applying various finetuning methods to the LLaMA-2 7B model. "A" indicate activation bits, "W" indicates weight bits. (c) Comparison multiplier and accumulator with PRESTE and other methods.

Fig. 1(c) provides a concise overview of various methods and our approach, revealing the trade-offs in arithmetic complexity and the feasibility of supporting fine-tuning. OliVe and Tender proposed custom number formats for efficient inference, but they are not applicable to fine-tuning [8, 18]. MXFP supports fine-tuning, its reliance on high-precision arithmetic limits the efficiency of fine-tuning. In addition, MXFP mitigates quantization errors, it does not fully preserve the dynamic range of conventional BF16 tensors. This limitation introduces errors in dot-product computations and degrades fine-tuning quality. For example, exponents below a certain threshold are truncated, introducing errors in dot-product computations and affecting overall accuracy.

To overcome these limitations, we propose **PRESTE (Preserving Precision of Tiny Exponents)**, an approach designed to retain all exponent values without loss, alongside a hardware architecture optimized for energy and area efficiency. Unlike traditional FP8 approaches which require FP32 accumulator, PRESTE's computation unit utilizes 4-bit integer (INT4) multiplier and INT21 accumulator, enabling area- and energy-efficient hardware implementation, without sacrificing numerical fidelity. We observe that, in many vectors, only a small subset of values have tiny exponents that are much smaller than the maximum exponent within the same vector. These small-magnitude values often vanish during quantization due to limited precision, leading to numerical errors during computation. By preserving values with these exponents, PRESTE minimizes quantization errors during computations. In addition, PRESTE efficiently supports a wide exponent range for activations and gradients, enabling to achieve Bfloat16 level accuracy, while using sub-8-bit precisions. This capability alleviates the memory traffic overhead of high-precision tensors and supports efficient computation, making it feasible to fine-tune LLMs directly on edge environments, as shown in Fig. 1.

In summary, our main contributions are as follows:

- We propose a method for fine-tuning that preserves the full range of exponent values without truncation. By preserving wide-range exponent elements during fine-tuning, such as activations and gradients, our method achieves the inference and fine-tuning accuracy level of Bfloat16.
- We introduce a processing architecture to address the limitations of FP8 dot-product operations, which require high-precision hardware component. Our architecture minimizes hardware overhead with the 21-bit integer accumulator, while still covering the entire range of 8-bit exponents.

- We validate the PRESTE quantization method and hardware design through extensive evaluations, applying PRESTE to fine-tune and inference across various LLMs and datasets. PRESTE achieves 2.22-3.55 $\times$ higher energy efficiency and 1.67-2.26 $\times$ higher performance per unit area.

2 Background and Challenges

2.1 Vanishing of Tiny Value During Training

The MXFP addresses the heuristic adjustment by sharing an 8-bit exponent across 32 elements. However, MXFP still suffers from significant quantization errors when a few elements in the vector have exponents that differ substantially from the maximum exponent. For instance, in MXFP8-E4M3 (4-bit exponent, 3-bit mantissa), the 4-bit exponent supports reliable quantization only when the exponent difference within a vector is less than or equal to 15, as indicated by the blue bars in Fig. 2. When the exponent difference exceeds 15, accurate representation becomes impossible, causing elements to be quantized as a subnormal or zero. In Fig. 2, orange and red bars indicate the proportion of the vectors that cannot be covered by conventional MXFP8-E4M3 and MXFP8-E5M2, respectively. This quantization leads to distortions in dot-product computations, particularly when subnormal or zero values are multiplied by large counterpart elements.

2.2 Preserving Exponents Precision in Dot-Product

Fig. 3 visualizes the relative errors in dot-product computations across various formats. The x-axis and y-axis represent the difference between the maximum and minimum exponents in each 32-element vector, A and B, respectively. The preservation of precision in tiny exponents ensures more accurate and reliable dot-product

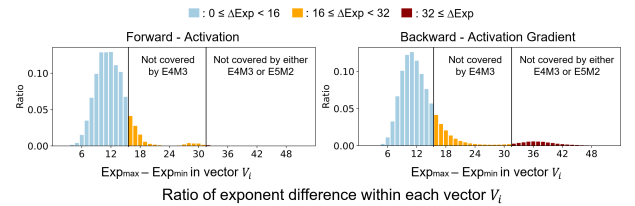


Figure 2: Distribution of the difference between the maximum and minimum exponents within 32 elements vector of activation and gradient tensors.

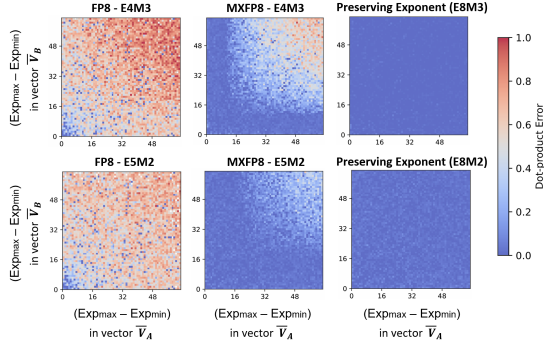


Figure 3: Visualization of the dot-product error for two vectors across various number formats. The x-axis and y-axis represent the difference between the maximum and minimum exponents in vectors A and B, respectively.

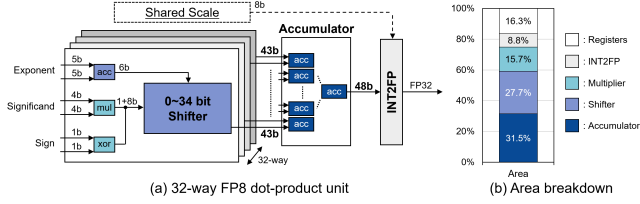


Figure 4: (a) The block diagram of a 32-way dot product unit for FP8. (b) The area breakdown of dot-product unit.

computations. By using an 8-bit exponent, number format can represent the same exponent range as Bfloat16 and FP32. Consequently, it prevents the truncation to subnormal or zero, preserving accuracy in the dot-product computation. Preserving tiny exponent precision prevents the truncation to subnormal or zero and ensures accurate computation. The key innovation of our proposed method lies in preserving the precision of an 8-bit exponent while utilizing only a 3-bit exponent representation.

2.3 FP8-based Dot-product Unit

As discussed earlier, effective fine-tuning requires both preserving wide exponent range to reduce quantization-induced errors and ensuring computational efficiency. In prior approaches, FP8-based arithmetic is used to general matrix multiplication (GEMM) operation during training [23, 32]. To improve computation efficiency, dot-product unit typically performs multiple multiplications in parallel and the results are aggregated through a single FP accumulator [16]. Fig. 4(a) illustrates a 32-way FP8 dot-product unit. This unit first aligns the 32 multiplication results based on their respective exponents and then accumulates them into a final FP32 output. As shown in Fig. 4(a), the MXFP8 utilizes the FP8 as the element format, and requires additional registers to store the shared scaling factor in dotted line, without modification of the existing hardware [22, 27].

The FP8 hardware supports both FP8-E5M2 and FP8-E4M3 [4, 38]. To handle a wide range of exponents, the compute unit requires 34-bit shifter for alignment and a 43-bit adder tree for accumulation. This design achieves FP32-level precision, but suffers from a significant hardware cost [12, 20]. We analyzed the area overhead of the

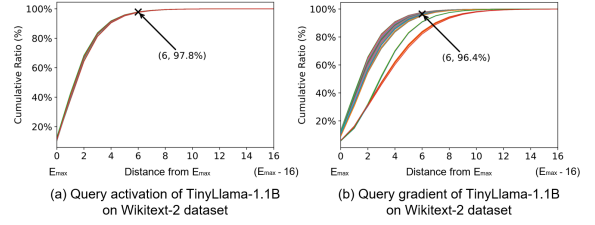


Figure 5: Cumulative ratio of exponent distances from maximum exponent within vector. E_{max} denotes maximum exponent.

shifter and accumulator in the MXFP8 dot-product unit. As shown in Fig. 4(b), the shifter accounts for 27.7% of the total area, while the accumulator occupies 31.5% and the pipeline registers contribute 16.3% of the area. These findings suggest that limiting the exponent range in the dot-product unit can reduce the size of the alignment shifter and accumulator, thereby improving hardware efficiency.

3 PRESTE Format

3.1 Representing 8-bit exponents with 3-bit

In many vectors, the range between the maximum and minimum exponents can be large. However, we observe that only a small fraction of elements contribute to this range. Most elements have exponents close to the maximum in each vector, indicating that the exponent values are tightly clustered. This localized distribution suggests that reduced bit precision can be applied with minimal overhead.

Fig. 5 presents the profiling results for the exponents of the query during fine-tuning of the LLaMA-2 7B model on the WikiText-2 dataset using Bfloat16. The x-axis represents the difference between each element's exponent and the max exponent within its 32-element vector. As shown in Fig. 5, 97.8% of activation and 96.4% of gradient exhibit an exponent difference of 6 or less, indicating that most elements are clustered around the maximum exponent within their respective vectors. We refer to these elements as normal elements in the rest of the paper. In the proposed scheme, we leverage this property and represent the difference between most elements' exponents and the maximum exponent using just 3-bit. Elements with exponent difference of 7 or more from the maximum exponent in each vector are classified as tiny elements. We assign 8-bit exclusively for the tiny exponents to enable precise representation across a wide exponent range. This strategy allows for efficient usage of bit precision, with most exponents requiring only 3-bit. A detailed analysis of the exponent distribution during fine-tuning with models on different datasets is provided in Sec 5.3.

3.2 PRESTE Number Format Encoding

Fig. 6(a) illustrates an example of a typical Bfloat16-format vector. In the proposed PRESTE format, as shown in Fig. 6(b), the element with the maximum exponent in the Bfloat16 is set as the **maximum exponent** E_{max} for the entire vector, and the difference between maximum exponent E_{max} and each element's exponent is calculated. Depending on the magnitude of the difference, the exponent difference is expressed by the 3-bit align exponent for normal elements or 8-bit tiny exponent for tiny elements.

For example, in Fig. 6, if the maximum exponent in a vector is $E_{max} = 141$, and the first element has an original exponent

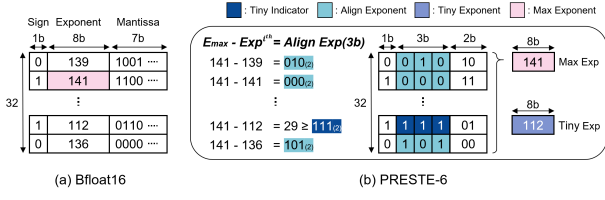


Figure 6: (a) The illustration of Bfloat16 data format. (b) The proposed PRESTE-6 data format with tiny and maximum exponents.

$Exp^1 = 139$, the 3-bit align exponent E_{align}^1 becomes:

$$E_{align}^1 = E_{max} - Exp^1 = 141 - 139 = 2 = 010_2$$

On the other hand, the 31st element has an original exponent $Exp^{31} = 112$, and its aligned exponent E_{align}^{31} becomes larger than 6:

$$E_{align}^{31} = E_{max} - Exp^{31} = 141 - 112 = 29.$$

In case $E_{align}^{31} > 6$, the 3-bit align exponent is used as a tiny indicator with value of 7 ($=111_2$), and this element is using a separate 8-bit tiny exponent E_{tiny} with value:

$$E_{tiny}^{31} = Exp^{31} = 112.$$

In summary, PRESTE has 3-bit align exponents to indicate distance from max exponent. In each case when the difference exceeds 6, 3-bit align exponent serves as tag, indicating that the element is a tiny elements, while the original exponent is stored as 8-bit tiny exponent. Multiple tiny elements may appear in a single vector, although they are infrequent in practice, accounting for approximately 1 in 32 elements in our observations. PRESTE format can be extended to different bit precision by assigning different number of bits to mantissa while using 3-bit align exponent in common. For example, PRESTE-6 use 2-bit for mantissa and PRESTE-8 uses 4-bit for mantissa. Both of them use 1-bit for sign, and 3-bit for align exponents.

3.3 Encoder and Memory Layout

The encoder supports quantization of floating-point vector into PRESTE vector with maximum exponent and tiny exponents. Fig. 7(a) shows the details of the encoder. The subtract unit computes the difference between maximum exponent (E_{max}) and each element's exponent (Exp^n). After the computation, PRESTE-6 rounds mantissa (Man^n) to 2-bit and PRESTE-8 rounds Man^n to 4-bit. The value (E^N) determines align exponent (E_{align}^N). In case of E^N is greater than 6, corresponding 8-bit tiny exponent E_{tiny} stored. The final vector consists of 32 PRESTE elements in reduced bit-precision, with a maximum exponent and tiny exponents. This approach efficiently quantizes vectors into the PRESTE format, covering the 8-bit exponent range with reduced bit precision.

As shown in Fig. 7(b), the PRESTE vector and its corresponding maximum exponent form the minimal unit of data in the layout. In the PRESTE-6 format, each vector unit occupies 25 bytes. These vectors are grouped into a PRESTE tile, which is stored in the local buffer. For example, an operand matrix with 1K vectors has a PRESTE tile that contains collection of vector groups along with the maximum exponent of each vector. The tiny data tile contains

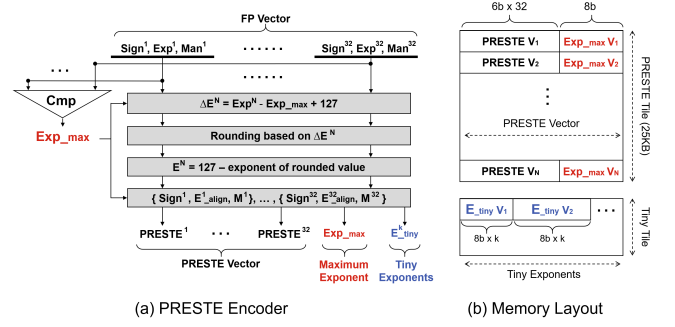


Figure 7: (a) The detailed architecture of PRESTE encoder. (b) Memory layout for PRESTE data tile and tiny data tile.

tiny exponents of corresponding PRESTE tile. Both the PRESTE tile and the tiny tile are stored in the local buffer, and a controller orchestrates tile mapping across the memory hierarchy.

4 PRESTE Architecture

4.1 Overall Architecture

Fig. 8 illustrates the overall architecture of PRESTE. Every buffers serve as a scratchpad that are explicitly managed by controllers. The lane consists of compute array operating in weight stationary dataflow, scheduler, vector processing unit (VPU), and encoder. The scheduler in each lane feeds activations and weights to the compute array, ensuring correct timings for efficient computation. In each cycle, the compute array generates FP results, which are either processed by the VPU or encoded back into the PRESTE format by the encoder. The computation results are then stored in DRAM through the buffer hierarchy, ensuring efficient data flow.

4.2 Dot-product Engine

4.2.1 Bit Precision of Dot-product Engine. As discussed in Sec. 2.3, the MXFP8 dot-product unit requires a 34-bit shifter for alignment and a 43-bit accumulator, which results in significant area overhead. In contrast, PRESTE normal elements use align exponent range between 0 and 6. Therefore, the dot-product engine (DPE) for normal elements can align multiplication results using a 12-bit shifter and accumulate with 21-bit accumulator. However, tiny elements employ 8-bit tiny exponent, necessitating separate shifter and accumulator to attain FP32-level accumulation.

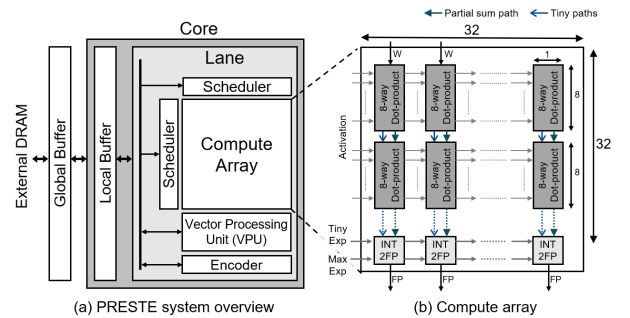


Figure 8: (a) The overview of PRESTE system architecture. (b) The internal structure of compute array.

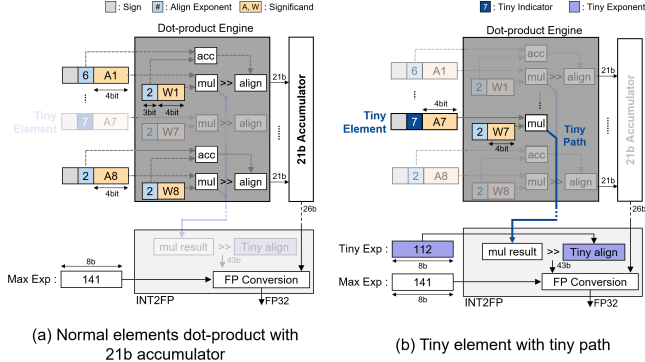


Figure 9: Simultaneous dot-product operation for the normal (a) and tiny elements (b) within in same dot-product unit.

4.2.2 Computation of Normal and Tiny Elements in a Unified DPE. To address these issues, PRESTE integrates dedicated tiny paths within each DPE, allowing the engine to process both normal and tiny elements simultaneously within a unified unit. Fig. 9(a) shows how DPE processes normal elements. Each significand participates in multiplication within the 8-way dot-product unit, and the engine adds the corresponding align exponents to determine alignment. The DPE aligns the multiplication result using 12-bit shifters and accumulates the results into 21-bit accumulator to generate a partial sum. The DPE passes down the partial sum to the next DPE in southward.

Fig. 9(b) illustrates how the DPE handles tiny elements simultaneously processing normal elements. When an element is identified as a tiny element, the multiplication result bypasses the alignment. The tiny result is routed through the dedicated tiny path to the INT2FP unit, which aligns the result based on its tiny exponent. After the alignment, FP conversion unit merges the tiny result with the normal partial sum to generate the final FP32 output. By eliminating intermediate truncation or rounding throughout the accumulation, PRESTE DPE maintains the FP32-level exponent range in the final output using 21-bit accumulator.

4.2.3 Intra-DPE Paths and Inter-DPE Paths. The bypass mechanism using tiny paths for tiny results reduces the computation bit width required within the DPE. However, assigning a dedicated tiny path to each element in 32-element dot-product introduces significant routing complexity.

To address this issue, we introduce two levels of tiny path: inter-DPE tiny paths, which are shared across DPEs, and intra-DPE tiny paths, which handle individual multiplication results within each DPE. [17]. Fig. 10(a) illustrates the microarchitecture of an 8-way DPE that includes two intra-DPE and two inter-DPE tiny paths. The multiplication result, either enters the shifter or routes through the intra-DPE tiny path based on the tiny tag, which is generated by 3-bit align exponent. The dashed inter-DPE paths collect tiny results from both intra-DPE paths and upper DPE, forward them to the downstream DPE. As shown in Sec. 3.1, tiny elements typically appear only once per 32-element vector. This low occurrence rate allows us to use a small number of tiny paths, limiting overhead.

4.2.4 Limiting the Number of Tiny Elements in Each Vector. To prevent collisions with random number of tiny elements across

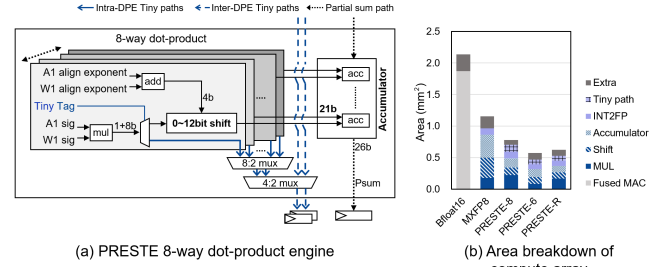


Figure 10: (a) The detailed diagram of dot-product engine. (b) The area breakdown of compute array.

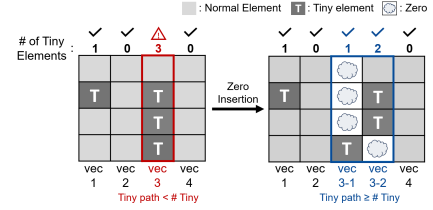


Figure 11: Zero insertion to limit the number of tiny elements in each vector.

inter-DPE tiny paths, a scheduler limits the number of tiny elements in each input vector before transmitting data to the compute array. As illustrated in Fig. 11, the scheduler ensures that each input vector contains no more than N tiny elements per DPEs with N inter-DPE tiny paths. Zero insertion is applied to fill the remaining positions in the sub-vectors, effectively mitigating inter-DPE tiny path collisions. The impact of this restriction is minimal because the occurrence of tiny elements exceeding the available tiny paths is rare. A detailed analysis of the performance impact due to the zero insertion caused by tiny elements is presented in Sec. 5.3.

4.3 Reconfigurable DPE

We also propose a minimally modified design to improve PRESTE-8 dot-product efficiency based on the existing PRESTE-6 DPE. This enhancement enables PRESTE-8-based GEMM operations in the transformer attention scores, supporting precise fine-tuning. In the PRESTE-6 format (sign, 3-bit align exponent, 2-bit mantissa), values are expressed as follows :

$$\langle s, (e_1, e_2, e_3), (m_1, m_2) \rangle$$

Before transmitting PRESTE-6 values into the compute array, we apply a 1-bit pre-align to the significand (including hidden bit) using the LSB of the 3-bit align exponent. The pre-align scheme converts 3-bit significand to a 4-bit pre-aligned significand, represented by:

$$\langle s, (e_1, e_2), [1.(m_1, m_2) \gg e_3] \rangle$$

More importantly, this design enables a multiplication in the PRESTE-8 format using 4 PRESTE-R computation units. This reconfigurability is useful to enhance the accuracy of LLMs by selectively applying PRESTE-8 format for multi-head attention computations during inference and fine-tuning, without requiring independent PRESTE-8 computational units. Since multi-head attention accounts for a relatively small proportion of the overall computation, using multiple PRESTE-R units to conditionally support PRESTE-8 does not incur noticeable degradation in overall computing performance. Fig. 10(b) shows the area breakdown for compute array consist of

Table 1: Fine-tuning results for different models on CSQA and perplexity across various quantization formats. The number in parentheses indicates the accuracy improvement compared with BF16 inference. "A" is the activation, "W" is the weight.

	Format	Bit		CSQA ↑				Perplexity ↓			
		A	W	TinyLlama 1.1B	StableLM 3B	LLaMA-2 7B	StableLM-2 12B	TinyLlama 1.1B	StableLM 3B	LLaMA-2 7B	StableLM-2 12B
Inference	BF16	16	16	40.17 (+0.00%)	51.28 (+0.00%)	55.48 (+0.00%)	60.88 (+0.00%)	14.69 (+0.00%)	52.40 (+0.00%)	9.48 (+0.00%)	6.32 (+0.00%)
LoRA	BF16	16	16	42.16 (+4.94%)	52.62 (+2.62%)	57.71 (+4.30%)	62.62 (+2.86%)	8.53 (-41.92%)	10.49 (-79.98%)	5.28 (-44.27%)	6.21 (-1.81%)
	MXFP-8	8	8	41.73 (+3.88%)	52.15 (+1.69%)	57.17 (+3.04%)	62.14 (+2.07%)	8.75 (-40.46%)	10.68 (-79.62%)	5.35 (-43.63%)	6.23 (-1.46%)
	PRESTE-8	8	8	42.10 (+4.81%)	52.57 (+2.52%)	57.68 (+3.97%)	62.42 (+2.53%)	8.64 (-41.88%)	10.55 (-79.88%)	5.29 (-44.18%)	6.20 (-1.86%)
	PRESTE-6	6	6	41.95 (+4.42%)	52.15 (+1.70%)	57.35 (+3.37%)	62.03 (+1.89%)	8.80 (-40.12%)	10.74 (-79.51%)	5.35 (-43.60%)	6.21 (-1.69%)
Inference	BF16	16	4	39.55 (+0.00%)	48.64 (+0.00%)	54.42 (+0.00%)	59.53 (+0.00%)	15.08 (+0.00%)	64.85 (+0.00%)	10.10 (+0.00%)	6.72 (+0.00%)
LoRA	BF16	16	4	40.86 (+3.32%)	50.55 (+3.99%)	56.05 (+3.00%)	62.22 (+4.52%)	9.20 (-38.99%)	11.06 (-82.95%)	5.55 (-45.06%)	6.22 (-7.41%)
	QLoRA	8	4*	40.59 (+2.63%)	50.31 (+3.43%)	55.73 (+2.41%)	61.98 (+4.12%)	9.33 (-38.13%)	11.21 (-82.71%)	5.54 (-45.15%)	6.23 (-7.21%)
	MXFP-8	8	4	40.67 (+2.85%)	50.36 (+3.54%)	55.67 (+3.20%)	61.87 (+3.92%)	9.29 (-38.39%)	11.23 (-82.86%)	5.56 (-44.93%)	6.29 (-6.37%)
	PRESTE-8	8	4	40.89 (+3.39%)	50.42 (+3.66%)	56.08 (+3.06%)	62.08 (+4.28%)	9.22 (-38.88%)	11.08 (-82.92%)	5.52 (-45.31%)	6.23 (-7.21%)
	PRESTE-6	6	4	40.73 (+3.00%)	50.21 (+3.24%)	55.77 (+2.48%)	61.82 (+3.84%)	9.30 (-38.36%)	11.20 (-82.74%)	5.57 (-44.86%)	6.29 (-6.37%)

MXFP-4 is used for 4-bit weight.
*NF4 is used for QLoRA weight.

32x32 MACs. PRESTE-R demonstrates high area efficiency, achieving area reduction of 3.50× and 1.89× compared to Bfloat16 and MXFP8, respectively.

Evaluation

5.1 Evaluation Methodology

5.1.1 Model Evaluation and Fine-tuning Dataset. To validate the effectiveness of the PRESTE method, we applied it to various pre-trained models, including TinyLlama-1.1B [45], StableLM-3B [37], LLaMA-2-7B [36], and StableLM-2-12B [1].

Perplexity (PPL) and CSQA score served as evaluation metrics for each LLM model. During evaluation, we used WikiText-2 dataset [21] to evaluate PPL using the *Hugging Face evaluate* [39]. To validate the effectiveness of PRESTE scheme across diverse datasets, we used the Alpaca dataset [35] and WikiText-2 for fine-tuning. CSQA score is evaluated on benchmarks such as Winogrande (Wino) [28], ARC_Easy (Arc_e), ARC_Challenge (Arc_c) [5], HellaSwag (Hella) [43], and OpenBookQA (OBQA) [34] using the *lm_eval* [7].

5.1.2 Fine-tuning Hyperparameters. During fine-tuning, we applied LoRA across all quantization methods [6, 41] and quantized every tensors except weight gradient and optimizer states for LoRA adapters [11]. For a fair comparison, the same hyperparameters are used across quantization methods for each model. The fine-tuning sequence length is set to 1024, with LoRA rank to 32. A learning rate of either 1e-4 or 1e-5 is applied. For the TinyLlama and LLaMA-2 are fine-tuned for 4.76 epochs on the WikiText-2 dataset and 1.63 epochs on the Alpaca dataset. For the StableLM series, fine-tuning was conducted for 5.53 epochs on WikiText-2 and 1.84 epochs on Alpaca dataset.

5.1.3 Quantization Baseline. We compared our PRESTE quantization method with BF16 and MXFP, a state-of-the-art (SOTA) numeric format optimized for LLM training.¹ Our evaluation focuses on approaches that do not rely on profiling-based heuristic adjustments, which are inadequate for edge device fine-tuning. We evaluated two scenarios for each model: expressing weights in MXFP4 format or the same number format as that of the activations. For both MXFP and PRESTE methods, we performed computations

for the multi-head attention in MXFP-8 and PRESTE-8 formats, respectively. Additionally, we evaluated QLoRA using NF4 format.

We also compared our scheme with SmoothQuant [40], OliVe [8] and Tender [18] for inference tasks. Although these methods are not compatible with training unlike our scheme, we still evaluate them to demonstrate the advantages of our PRESTE format in inference applications. For a fair comparison, we applied direct casting to the pre-trained models in PRESTE and MXFP formats, respectively.

5.1.4 Accelerator Baseline. We compared the energy consumption and performance of BF16, MXFP, and PRESTE under equivalent area constraints to evaluate hardware efficiency during fine-tuning. For QLoRA dequantizes NF4 weights to BF16, we adopted the same hardware configuration with BF16 baseline [6]. For MXFP, we used hardware optimized for dot-product operations [16, 20]. OliVe and Tender support MAC operations for custom number formats. Each PE supports 4-bit integer multiplication and 32-bit INT accumulation for 4-bit quantized data formats.

5.1.5 Hardware Implementation. All designs were implemented in Verilog and place and route were done using ICC2 with SRAM compiler on a 28nm process for area and power measurements with PrimePower. For cycle-accurate simulation, we integrate ScaleSim, LLMCompass for system simulation and DRAMsim3 for DRAM behavior, including tiny element overhead [19, 29, 44].

5.2 Accuracy Evaluation

5.2.1 Fine-tuning Accuracy. To demonstrate the effectiveness of our method in fine-tuning LLMs, we trained and evaluated various models on different datasets. Since PRESTE is designed to enable efficient fine-tuning on edge devices, we compared with MXFP, which supports direct casting without requiring heuristic parameter searches. The left side of Table 1 shows the results of fine-tuning each model on the Alpaca dataset, measuring their CSQA score. The right side of Table 1 shows the results of fine-tuning each model on the WikiText-2 dataset and measuring perplexity. PRESTE method with 8-bit precision consistently achieves better scores than MXFP at the same precision. Additionally, in most cases, the 6-bit PRESTE method outperforms 8-bit MXFP. This improvement stems from PRESTE's ability to preserve tiny exponents, ensuring precise exponent representation. By preserving exponents that are

¹Code available at <https://github.com/preste-git/PRESTE>

Table 2: Direct inference CSQA score on StableLM-3B across various quantization methods. The higher is better.

Method	Bit	A W	StableLM-3B					CSQA
			Hella.	Arc_c	Arc_e	Wino.	OBQA.	
BF16	16	4	52.61	37.37	65.32	61.48	26.40	48.64
SmoothQuant	8	4	29.01	21.93	37.31	58.99	15.40	32.53
OliVe	8	4	28.09	20.09	35.23	53.04	15.80	30.45
Tender	8	4	46.08	22.10	52.31	54.94	21.40	39.37
MXFP-8	8	4	51.95	35.49	63.30	61.96	26.40	47.82
PRESTE-8	8	4	52.57	36.69	64.86	62.90	26.20	48.64
PRESTE-6	6	4	52.08	36.01	64.86	61.17	25.80	47.98

*MXFP4 is used for 4-bit weight.

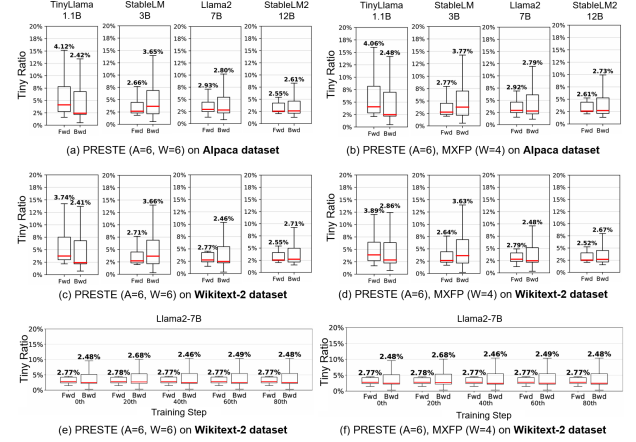
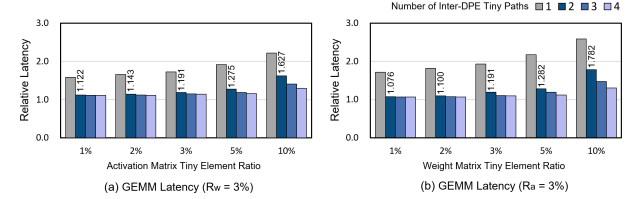
often compromised in conventional quantization methods, PRESTE significantly reduces errors in dot-product computations.

5.2.2 Inference Accuracy. Table 2 presents detailed inference scores on pre-trained StableLM-3B across various quantization methods without fine-tuning. The PRESTE method with 8-bit precision achieves better score than any other quantization methods at the same bit precision. Compared to inference-specialized formats, PRESTE demonstrates superior performance even on pre-trained models. This indicates that, even without fine-tuning, PRESTE achieves higher accuracy in downstream tasks.

5.3 Tiny Exponent Profiling

We intensively profiled the distribution of tiny elements of each tensor across LLM models with various datasets. All operand matrices including attention and linear modules are averaged for each matrix and are displayed as box plots. Fig. 12(a, b, c, d) provides an aggregated view of profiling data across different models with Alpaca and WikiText-2 dataset [21, 35]. Fig. 12(e, f) shows profiling results with different training steps with WikiText-2. Results were plotted every 20 steps on a total of 100 weight update steps, separately for forward and backward passes. The results indicate a broader distribution of tiny elements in the matrices associated with backward pass. Notably, even as the linear learning rate scheduler decreases the learning rate during fine-tuning, the proportion of tiny elements remains stable. **A consistent proportion of tiny elements is observed across forward and backward processes, regardless of variations in models, datasets and data precision.**

Based on these profiling results, we simulate the impact of tiny element distributions on the latency of GEMM operations in hardware including scheduling overhead. Fig. 13 compares the latency of GEMM operations on 4096×4096 activation and weight matrices during the fine-tuning of the LLaMA-2 7B model. The distribution of tiny elements in both operand matrices was profiled using data generated during fine-tuning. Latency was simulated for cases where tiny elements comprised 2%, 3%, 5%, and 10% of a matrix. The simulations considered various number of tiny paths for the first operand matrix, while the second operand matrix had a fixed 3% proportion of tiny elements and 2 tiny paths. The results indicate that with two tiny paths for each activation and weight matrices, the latency overhead for GEMM operations remains negligible, even as the proportion of tiny elements increases in the operand matrix. Based on these findings, PRESTE incorporates 2 tiny paths for each of activations and weights, ensuring efficient processing with minimal latency impact.

**Figure 12: Each plot shows the ratio of tiny elements in tensors during fine-tuning process of models on different datasets.****Figure 13: Relative latency of GEMM operation with various tiny elements ratio. R_w and R_a represent the ratio of tiny elements in weights and activations, respectively.**

5.4 Hardware Evaluation

5.4.1 Hardware Configuration. Table 3 summarizes the hardware configurations based on different compute types. PRESTE demonstrates high area efficiency in compute array, achieving area reductions of $3.51\times$ and $1.89\times$ compared to Bfloat16 and MXFP8, respectively. OliVe [8], which is designed for inference, is $1.44\times$ larger than PRESTE due to its 32-bit integer accumulation unit in each PE.

Therefore, to ensure a fair comparison under the same core area constraints, the number of lanes is adjusted accordingly across configurations. We also considered additional overhead introduced by other components such as the vector unit, scheduler, and encoder. Table 4 presents the area and power breakdown of each component in PRESTE single core including routing overhead.

5.4.2 Analysis on Fine-tuning. We analyze area efficiency and energy efficiency to run the fine-tuning across various models and numerical formats, with sequence lengths set to 1024 and batch sizes to 4. Fig. 14 illustrates the TOPS/ mm^2 and TOPS/W for forward and backward pass, normalized to Bfloat16 baseline for each

Table 3: Hardware Specification by Compute Type

	BF16	MXFP8	OliVe	Tender	PRESTE
Compute Array					
Area (mm^2)	2.136	1.154	0.878	0.636	0.610
Array / Lane	1	1	1	1	1
Lane / Core	2	3	4	5	5
Core Area (mm^2)	5.368	5.139	5.329	5.171	5.126
Common	2-channel DDR4 (16GB, 51.2GB/s) Global Buffer (4MB), Local Buffer (256KB), Core Frequency (500MHz)				

Table 4: Area and Power Breakdown of PRESTE Single Core

	Area (mm ²)	Power (mW)
Single Core	5.126	1274
Local Buffer	0.586 (11.4%)	0.2343pJ/bit
Lane	4.540	1274
Compute Array	3.049 (59.5%)	977 (76.7%)
Vector Processing Unit	1.278 (24.9%)	229 (17.9%)
Encoder + Scheduler	0.213 (4.2%)	69 (5.4%)

model. QLoRA employs same hardware with BF16, which results in identical evaluation output with baseline. The PRESTE configuration includes 2.5 $\times$ more number of compute unit than Bfloat16 configuration and 1.67 $\times$ more unit than MXFP, resulting in higher throughput. Moreover, decreased bit precision of PRESTE minimizes off-chip DRAM access, thereby lowering energy consumption. Overall, PRESTE achieves a 2.77 $\times$ TOPS/W improvement for the forward pass, and 2.58 $\times$ TOPS/W improvement for the backward pass.

5.4.3 Analysis on Inference. We also analyze area efficiency and energy efficiency during the inference, using a batch size of 8 with 1024 input tokens for each model. The evaluation of area efficiency and energy efficiency is based on the time taken to generate the 32th token. During inference, PRESTE achieves 1.28 $\times$ higher TOPS/mm² than Tender, which uses multiple 4-bit MACs to compute a single 8-bit format, as shown in Fig. 15. Unlike baseline accelerators such as Tender and OliVe, which support inference only, PRESTE supports both inference and fine-tuning while outperforming them in inference efficiency.

Table 5: Compute Throughput, Peak Memory and Off-chip Memory Access Traffic of Fine-tuning

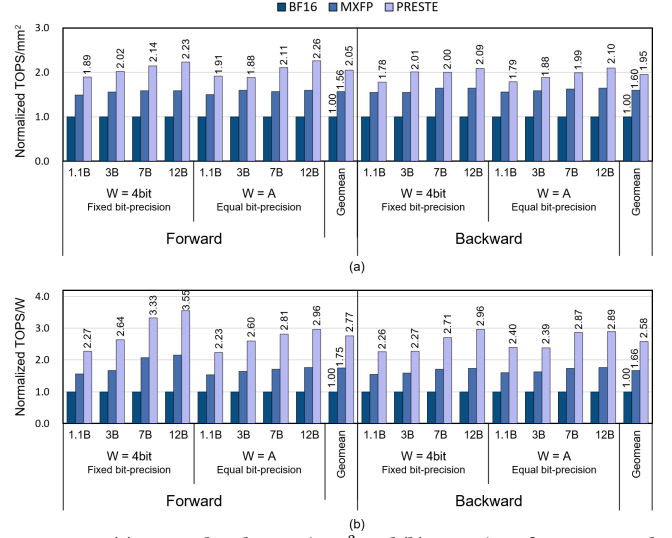
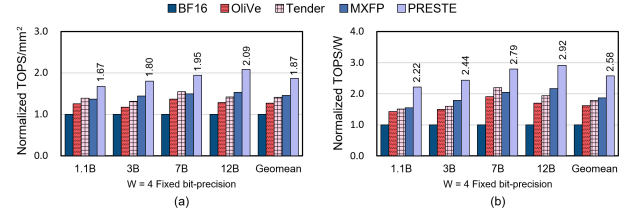
	Compute Throughput $\uparrow$		Peak Memory $\downarrow$		Off-chip Memory Access $\downarrow$	
	w/ check	w/o check	w/ check	w/o check	w/ check	w/o check
TinyLlama-1.1B	0.726	1.000	0.558	1.000 / 0.398*	1.412	1.000 / 0.4585*
StableLM-3B	0.712	1.000	0.602	1.000 / 0.394*	1.422	1.000 / 0.4402*
LLaMA2-7B	0.702	1.000	0.761	1.000 / 0.391*	1.428	1.000 / 0.4015*

*PRESTE is simulated result

6 Discussion

6.1 Activation Checkpointing

Activation checkpointing is a technique that reduces peak memory requirements during training by recomputing activation tensor during backward pass, which lowers activation storage overhead [3]. However, recomputing activations in backward pass incurs additional computation and off-chip memory traffic, resulting in a negative impact on overall performance [15]. Table 5 presents the normalized results of compute throughput, peak memory requirement and memory traffic during the finetuning process with A6000 GPU, a sequence length 1024 and a batch size 1 across various networks. Compared to fine-tuning without checkpointing, fine-tuning with activation checkpointing results in 0.712-0.726 $\times$ less compute throughput and 1.412-1.428 $\times$ more DRAM access. In contrast, applying quantization to high-precision activations and weights mitigates memory traffic. Therefore, applying PRESTE reduces memory access and increase compute throughput for fine-tuning in edge environments.


Figure 14: (a) Normalized TOPS/mm² and (b) TOPS/W of PRESTE and baseline designs during fine-tuning.

Figure 15: (a) Normalized TOPS/mm² and (b) TOPS/W of PRESTE and baseline designs during inference.

7 Conclusion

In this paper, we propose PRESTE, a novel quantization approach designed to preserve the precision of small exponents with the goal of enhancing both inference and fine-tuning accuracy. The key idea is to preserve exponents smaller than threshold that are often truncated during the quantization process, thereby reducing the error in dot-product computations and improving overall accuracy. PRESTE-based LLM models show inference and fine-tuning performance comparable to the Bfloat16-based LLM models. Additionally, we propose PRESTE hardware that reduces the overhead associated with alignments and accumulators in the conventional FP dot-product operations. PRESTE hardware achieves area efficiency improvement of 1.67-2.26 $\times$ and energy efficiency improvement of 2.22-3.55 $\times$ compared to Bfloat16 hardware for inference and fine-tuning process.

Acknowledgments

This work was supported in part by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (RS-2024-00395134, DPU-Centric Datacenter Architecture for Next-Generation AI Devices, RS-2026-25507427, Development of Efficient Architectures and Training Techniques for High-Performance Lightweight AI Models, RS-2023-00256081: artificial intelligence semiconductor support program to nurture the best talents, No. RS-2026-25549926: AI Semiconductor Innovation Research Center), and BK21 FOUR program and ISRC at Seoul National University. The EDA tool was supported by the IC Design Education Center (IDEC). (Corresponding Author: Jae-Joon Kim).

References

- [1] Marco Bellagente, Jonathan Tow, Dakota Mahan, Duy Phung, Maksym Zhuravinskiy, Reshith Adithyan, James Baicoianu, Ben Brooks, Nathan Cooper, Ashish Datta, et al. 2024. Stable LM 2 1.6 B Technical Report. *arXiv preprint arXiv:2402.17834* (2024).
- [2] Huangxu Chen, Yingbo Hao, Yi Zou, and Xinyu Chen. 2025. OA-LAMA: An Outlier-Adaptive LLM Inference Accelerator with Memory-Aligned Mixed-Precision Group Quantization. In *2025 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [3] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174* (2016).
- [4] Jack Choquette. 2023. Nvidia hopper h100 gpu: Scaling performance. *IEEE Micro* 43, 3, 9–17.
- [5] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457* (2018).
- [6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2024. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems* 36 (2024).
- [7] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2024. The Language Model Evaluation Harness. <https://doi.org/10.5281/zenodo.12608602>
- [8] Cong Guo, Jiaming Tang, Weiming Hu, Jingwen Leng, Chen Zhang, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. 2023. Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–15.
- [9] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for NLP. In *International conference on machine learning*. PMLR, 2790–2799.
- [10] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
- [11] Hyesung Jeon, Yulhwa Kim, and Jae-joon Kim. 2024. L4q: Parameter efficient quantization-aware training on large language models via lora-wise lsq. *arXiv preprint arXiv:2402.04902* (2024).
- [12] Himanshu Kaul, Mark Anders, Sanu Mathew, Seongjong Kim, and Ram Krishnamurthy. 2019. Optimized fused floating-point many-term dot-product hardware for machine learning accelerators. In *2019 IEEE 26th Symposium on Computer Arithmetic (ARITH)*. IEEE, 84–87.
- [13] Diederik P Kingma. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [14] Jahyun Koo, Dahoon Park, Sangwoo Jung, and Jaeha Kung. 2024. OPAL: Outlier-Preserved Microscaling Quantization Accelerator for Generative Large Language Models. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
- [15] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems* 5 (2023), 341–353.
- [16] Tomás Lang and Javier D Bruguera. 2002. Floating-point fused multiply-add with reduced latency. In *Proceedings. IEEE International Conference on Computer Design: VLSI in Computers and Processors*. IEEE, 145–150.
- [17] Jehun Lee and Jae-Joon Kim. 2025. Integer Unit-Based Outlier-Aware LLM Accelerator Preserving Numerical Accuracy of FP-FP GEMM. In *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–7.
- [18] Jungi Lee, Wonbeom Lee, and Jaewoong Sim. 2024. Tender: Accelerating Large Language Models via Tensor Decomposition and Runtime Requantization. *arXiv preprint arXiv:2406.12930* (2024).
- [19] Shang Li, Zhiyuan Yang, Dhiraj Reddy, Ankur Srivastava, and Bruce Jacob. 2020. DRAMsim3: A cycle-accurate, thermal-capable DRAM simulator. *IEEE Computer Architecture Letters* 19, 2 (2020), 106–109.
- [20] David R Lutz, Anisha Saini, Mairin Kroes, Thomas Elmer, and Harsha Valsaraju. 2024. Fused FP8 4-Way Dot Product With Scaling and FP32 Accumulation. In *2024 IEEE 31st Symposium on Computer Arithmetic (ARITH)*. IEEE, 40–47.
- [21] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer Sentinel Mixture Models. *arXiv:1609.07843 [cs.CL]*
- [22] Paulius Micikevicius, Stuart Oberman, Pradeep Dubey, Marius Cornea, Andres Rodriguez, Ian Bratt, Richard Grisenthwaite, Norm Jouppi, Chiachen Chou, Amber Huffman, et al. 2023. OCP 8-bit Floating Point Specification (OFP8). *Open Compute Project* (2023).
- [23] Paulius Micikevicius, Dusan Stolic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, et al. 2022. Fp8 formats for deep learning. *arXiv preprint arXiv:2209.05433*.
- [24] Microsoft. 2024. MX Pytorch Emulation Library. <https://github.com/microsoft/microscaling>
- [25] Houwen Peng, Kan Wu, Yixuan Wei, Guoshuai Zhao, Yuxiang Yang, Ze Liu, Yifan Xiong, Ziyue Yang, Bolin Ni, Jingcheng Hu, et al. 2023. Fp8-lm: Training fp8 large language models. *arXiv preprint arXiv:2310.18313*.
- [26] Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. 2020. Adapterfusion: Non-destructive task composition for transfer learning. *arXiv preprint arXiv:2005.00247*.
- [27] Bitá Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, et al. 2023. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537* (2023).
- [28] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. Winogrande: An adversarial winograd schema challenge at scale. *Commun. ACM* 64, 9 (2021), 99–106.
- [29] Ananda Samajdar, Yuhao Zhu, Paul Whatmough, Matthew Mattina, and Tushar Krishna. 2018. Scale-sim: Systolic cnn accelerator simulator. *arXiv preprint arXiv:1811.02883*.
- [30] Yijia Shao, Tianshi Li, Weiyan Shi, Yanchen Liu, and Diyi Yang. 2024. Privacylens: Evaluating privacy norm awareness of language models in action. *Advances in Neural Information Processing Systems* 37 (2024), 89373–89407.
- [31] Karan Singhal, Shekoofeh Azizi, Tao Tu, S Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen Pfohl, et al. 2023. Large language models encode clinical knowledge. *Nature* 620, 7972 (2023), 172–180.
- [32] Xiao Sun, Jungwook Choi, Chia-Yu Chen, Naigang Wang, Swagath Venkataramani, Vijayalakshmi Viji Srinivasan, Xiaodong Cui, Wei Zhang, and Kailash Gopalakrishnan. 2019. Hybrid 8-bit floating point (HFP8) training and inference for deep neural networks. *Advances in neural information processing systems* 32.
- [33] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. 2020. Test-time training with self-supervision for generalization under distribution shifts. In *International conference on machine learning*. PMLR, 9229–9248.
- [34] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. 2018. Commonsenseqa: A question answering challenge targeting commonsense knowledge. *arXiv preprint arXiv:1811.00937* (2018).
- [35] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Stanford alpaca: An instruction-following llama model.
- [36] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shrutli Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- [37] Jonathan Tow, Marco Bellagente, Dakota Mahan, and Carlos Riquelme. 2024. StableLM 3B 4E1T. <https://huggingface.co/stabilityai/stablelm-3b-4e1t>
- [38] Swagath Venkataramani, Vijayalakshmi Srinivasan, Wei Wang, Sanchari Sen, Jintao Zhang, Ankur Agrawal, Monodeep Kar, Shubham Jain, Alberto Mannari, Hoang Tran, et al. 2021. RaPiD: AI accelerator for ultra-low precision training and inference. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 153–166.
- [39] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*. 38–45.
- [40] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*. PMLR, 38087–38099.
- [41] Yuhui Xu, Lingxi Xie, Xiaotao Gu, Xin Chen, Heng Chang, Hengheng Zhang, Zhengsu Chen, Xiaopeng Zhang, and Qi Tian. 2023. Qa-lora: Quantization-aware low-rank adaptation of large language models. *arXiv preprint arXiv:2309.14717*.
- [42] Ning Yang, Zongwu Wang, Qingxiao Sun, Liqiang Lu, and Fangxin Liu. 2025. PISA: Efficient Precision-Slice Framework for LLMs with Adaptive Numerical Type. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–7.
- [43] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830* (2019).
- [44] Hengrui Zhang, August Ning, Rohan Baskar Prabhakar, and David Wentzlaff. 2024. Llmcompass: Enabling efficient hardware design for large language model inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1080–1096.
- [45] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385* (2024).